

EXPLORING COLUMNAR EXECUTION IN DUCKDB THROUGH A SALES ANALYTICS APPLICATION

Yunhui Dong

INTRODUCTION

- Built a sales analytics application using DuckDB and Python
- Goal: connect DuckDB internals to analytical application behavior
- Dataset: supermarket sales data

CORE INTERNAL FOCUS AREAS

- Columnar storage
- Vectorized query execution
- DuckDB is designed for analytical / OLAP workloads
- Good fit for scans, filtering, and grouped aggregation

APPLICATION OVERVIEW

- Main table: supermarket_sales
- Key columns: city, product_line, sales, sale_date, payment
- Main operations:
 - total sales by city
 - sales in a date range
 - average sales by product line

	invoice_id	branch	city	customer_type	gender	product_line	unit_price	quantity	tax_5_percent	sales	sale_date	sale_time	payment	cogs	gross_margin_percentage	gross_income	rating
0	750-67-8428	Alex	Yangon	Member	Female	Health and beauty	74.69	7	26.1415	548.9715	2019-01-05	01:08:00	Ewallet	522.83	4.761905	26.1415	9.1
1	226-31-3081	Giza	Naypyitaw	Normal	Female	Electronic accessories	15.28	5	3.8200	80.2200	2019-03-08	10:29:00	Cash	76.40	4.761905	3.8200	9.6
2	631-41-3108	Alex	Yangon	Normal	Female	Home and lifestyle	46.33	7	16.2155	340.5255	2019-03-03	01:23:00	Credit card	324.31	4.761905	16.2155	7.4
3	123-19-1176	Alex	Yangon	Member	Female	Health and beauty	58.22	8	23.2880	489.0480	2019-01-27	08:33:00	Ewallet	465.76	4.761905	23.2880	8.4
4	373-73-7910	Alex	Yangon	Member	Female	Sports and travel	86.31	7	30.2085	634.3785	2019-02-08	10:37:00	Ewallet	604.17	4.761905	30.2085	5.3

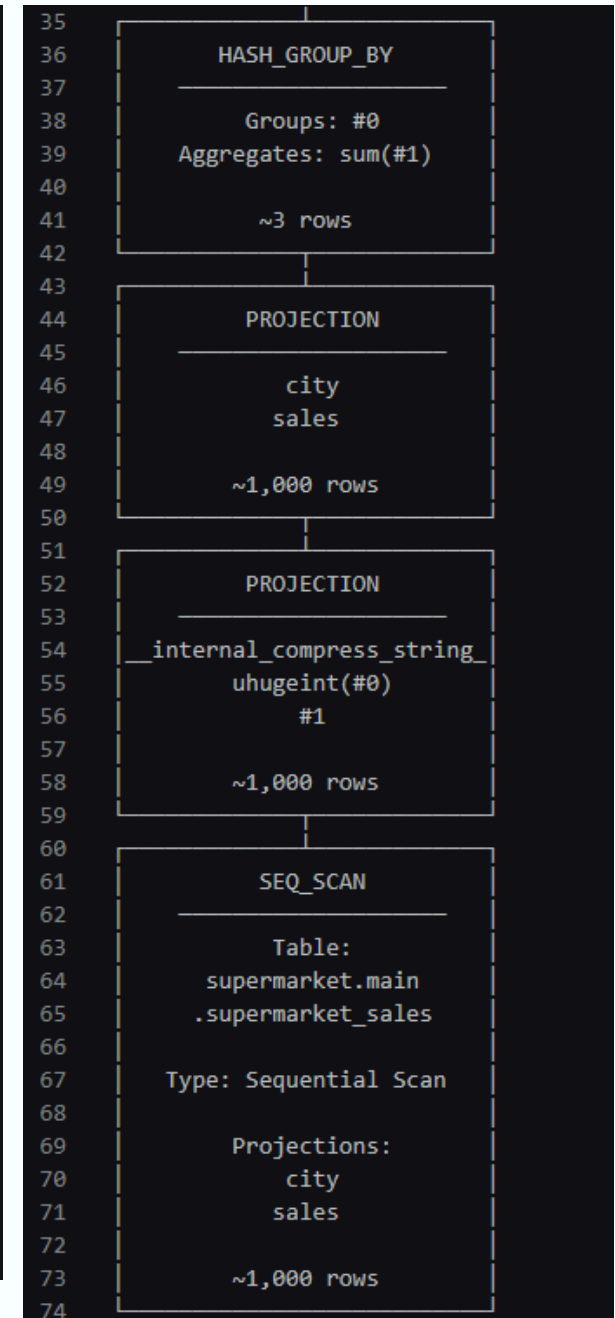
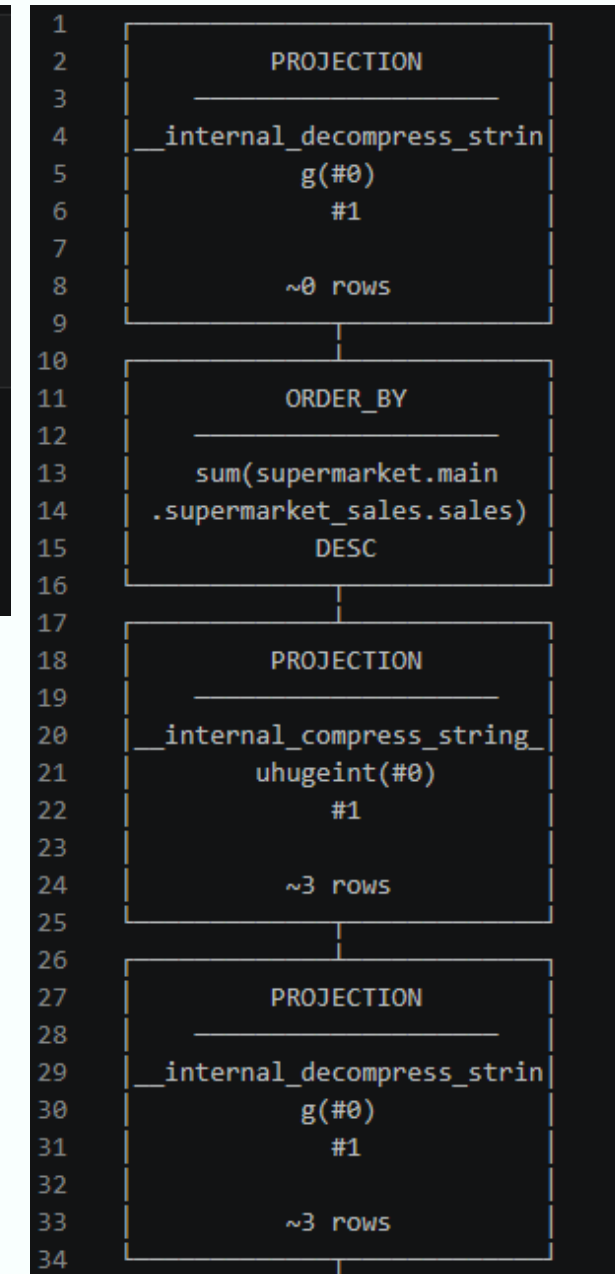
MAJOR OPERATION 1: TOTAL SALES BY CITY

- What the application does
 - Computes total sales for each city
- What the database does internally
 - Reads city and sales
 - Performs grouped aggregation
 - EXPLAIN shows HASH_GROUP_BY
- Why that internal behavior matters
 - The query does not need all columns in the table
 - This shows how DuckDB supports analytical aggregation efficiently

```
q1 = """
SELECT city, SUM(sales) AS total_sales
FROM supermarket_sales
GROUP BY city
ORDER BY total_sales DESC;
"""

df_q1 = con.execute(q1).df()
df_q1
```

	city	total_sales
0	Naypyitaw	110568.7065
1	Yangon	106200.3705
2	Mandalay	106197.6720



Sequential scan of city and sales → hash-based grouping by city → sum sales for each group → sort results by total sales descending

MORE MAJOR OPERATIONS AND INTERNAL MAPPINGS

Operation 2: Sales in a Date Range

- Application: computes total sales in a selected time window
- Database: filters rows on *sale_date* and aggregates *sales*
- Why it matters: shows how filtering and aggregation support reporting workloads

Operation 3: Average Sales by Product Line

- Application: compares product categories by average sales
- Database: groups by *product_line* and aggregates sales
- Why it matters: shows another analytical query pattern supported by DuckDB

VALIDATION AND CONCLUSION

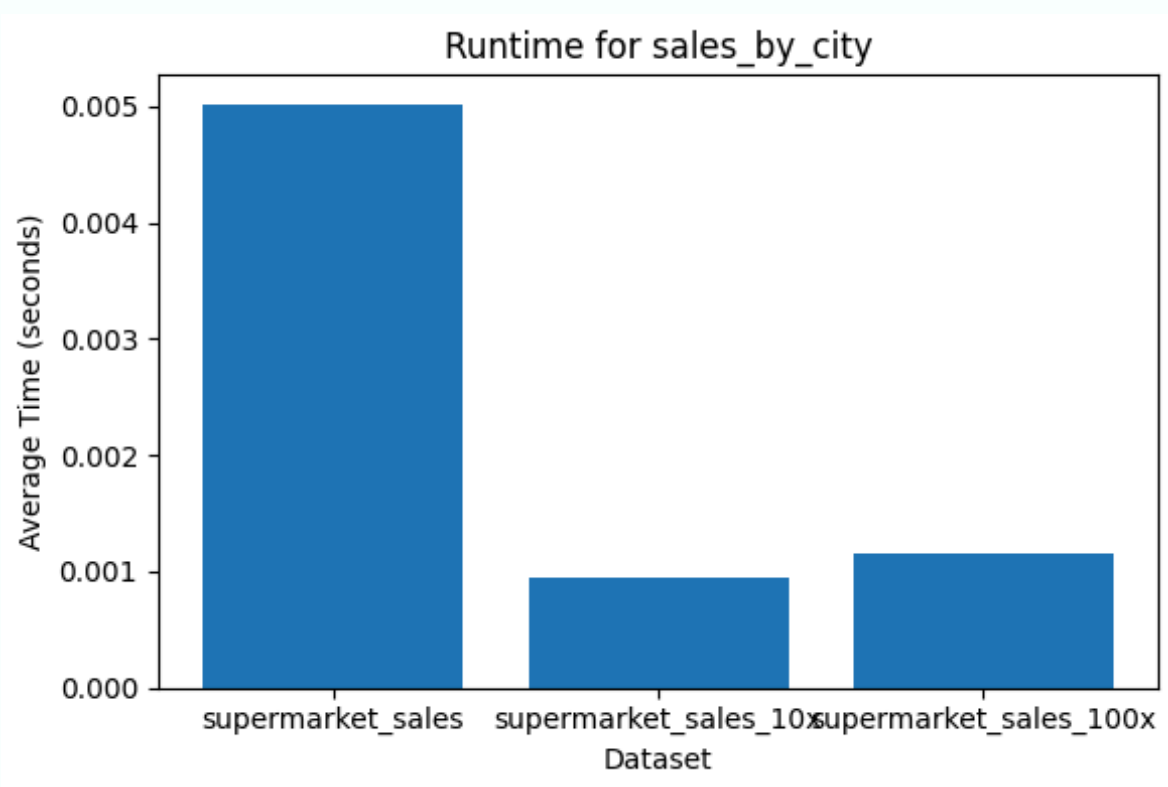
Validation

- Used EXPLAIN and EXPLAIN ANALYZE to inspect query behavior
- Created scaled datasets: original / 10x / 100x
- Compared runtime for representative analytical queries

Key Takeaways

- DuckDB is a good fit for analytical workloads
- Grouped aggregation and filtering map naturally to its execution model
- The application demonstrates how database internals affect query behavior

	table	query	avg_time_sec
0	supermarket_sales	sales_by_city	0.005016
1	supermarket_sales	avg_sales_by_product	0.001540
2	supermarket_sales	january_sales	0.001316
3	supermarket_sales_10x	sales_by_city	0.000943
4	supermarket_sales_10x	avg_sales_by_product	0.001218
5	supermarket_sales_10x	january_sales	0.000410
6	supermarket_sales_100x	sales_by_city	0.001155
7	supermarket_sales_100x	avg_sales_by_product	0.001945
8	supermarket_sales_100x	january_sales	0.002141





**THANK
YOU**